SHERPA PROS · EDITORIAL

Sherpa Pros — Investor Metrics Dashboard Design

DATE: 2026-04-22 **OWNER:** PHYROM (DATA-ANALYTICS-REPORTER AGENT DRAFT) **AUDIENCE:** INVESTORS (DATA ROOM LINK), PHYROM (OPERATING DASHBOARD), BETA-COHORT REVIEW **ROUTE:** `/ADMIN/INVESTOR-METRICS` (FILE: `SRC/APP/(DASHBOARD)/ADMIN/INVESTOR-METRICS/PAGE.TSX`) **STATUS:** V1 — PHASE 0. MOST METRICS SURFACE MOCKED / EXTRAPOLATED VALUES UNTIL BETA COHORT DATA FLOWS. EACH WIDGET HAS A CLEAR `// TODO` SWAP POINT.

1. Purpose

This dashboard is the **live-traction artifact** referenced in spec §6.7 ("Live-metrics dashboard link" — Nice to have by W8) and the source of truth for every metric named in spec §5.5 (Traction metrics for pitch deck) and §12 (Phase gates).

It serves three stakeholders simultaneously:

1. **Investors** — link in the data room; updates in real time as beta cohort transacts. Removes the "trust-me-bro" smell from the deck.
2. **Phyrom** — operating dashboard; spot trends, react to liquidity dips, time outreach campaigns.
3. **Beta-cohort review** — single page Phyrom can screen-share on weekly 10-min beta calls.

Constraints baked in:

- **No "Wiseman" branding externally** — but this is internal admin, so internal name allowed (per CLAUDE.md and spec §3.3).
- **Spell out abbreviations** (per `feedback_no_contractor_jargon.md`) — GMV → "Gross Merchandise Value", NPS → "Net Promoter Score" on first hover/legend.
- **Mobile-first** but investor view assumed desktop-primary; responsive breakpoints from Tailwind defaults.

2. Tech stack

LAYER	CHOICE	WHY
Framework	Next.js 16 App Router (Server Component default)	Matches CLAUDE.md convention; lets queries run server-side; no client bundle bloat for static numbers.

Charts	**Tremor** (`@tremor/react`)	Purpose-built for dashboards. Comes with `AreaChart`, `BarChart`, `LineChart`, `DonutChart`, `Tracker`, `Metric`, `BadgeDelta`, `ProgressBar`, sparklines. Works with Tailwind 4.
Tables	**TanStack Table** (`@tanstack/react-table`)	Headless; pairs with shadcn for styling; needed for per-metro / per-trade breakouts.
Layout primitives	**shadcn/ui** (Card, Tabs, Badge, Separator)	Gives Tremor a consistent surrounding shell that matches the rest of the admin.
Data	Drizzle ORM + Neon Postgres via `src/db/connection.ts` `query()` and `sql` helpers	Same pattern as every other admin page.
Auth	`requireRole('client' | 'pro' | 'pm' | 'tenant')` — **need a `requireAdmin()` helper later**. For Phase 0, gate on Phyrom's specific Clerk user ID via env `ADMIN_USER_IDS` (comma-separated).	

Install commands needed

```
npm install @tremor/react @tanstack/react-table
npm install class-variance-authority clsx tailwind-merge lucide-react tailwindcss-animate
# shadcn/ui — initialize in repo root, choose "default" style, "neutral" base color
npx shadcn@latest init
npx shadcn@latest add card tabs badge separator
```

Tremor v3 also requires a small Tailwind config addition (color tokens). With Tailwind 4's CSS-first config, those tokens go into `src/app/globals.css` under `@theme` — coordinator should verify post-install.

3. Page layout

```
┌──────────────────────────────────────────────────────────────────┐
│  Header: "Sherpa Pros — Live Traction"                           │
│  Sub: "Beta cohort metrics. Updates every 5 min. As of {timestamp}" │
│  Cohort phase pill: "PHASE 0 · BETA · {n} active pros"           │
├──────────────────────────────────────────────────────────────────┤
│  Row 1 — Headline metrics (4-up grid, KPI cards)                 │
│  [GMV total]  [Take-rate $]  [Active pros]  [NPS pro / client]   │
├──────────────────────────────────────────────────────────────────┤
│  Row 2 — Tabs: 7d / 30d / 90d window selector (shared across row 3) │
│  Row 3 — Charts (2-up)                                           │
│  [GMV time-series, area chart]   [Take-rate capture, line chart] │
├──────────────────────────────────────────────────────────────────┤
│  Row 4 — Liquidity & speed                                       │
│  [Pro liquidity ratio bar by metro] [Avg bids per job + sparkline] │
├──────────────────────────────────────────────────────────────────┤
│  Row 5 — Cohort & retention                                      │
│  [Pro retention cohort table]      [Client repeat rate progress] │
├──────────────────────────────────────────────────────────────────┤
│  Row 6 — NPS time series (full-width line)                       │
├──────────────────────────────────────────────────────────────────┤
│  Row 7 — Wiseman usage (3-up sparkline cards)                    │
│  [Quote validations] [Code checks] [Scope approvals]             │
├──────────────────────────────────────────────────────────────────┤
│  Row 8 — Funnel (full-width, stepped horizontal bar)             │
│  posted → matched → bid → accepted → completed → paid → reviewed │
├──────────────────────────────────────────────────────────────────┤
│  Footer: "Phase 0 gate progress: $250K non-dilutive committed    │
│           OR $500K Wefunder + angels OR Tier-1 accelerator. {%}" │
└──────────────────────────────────────────────────────────────────┘
```

4. Widgets

For each: source tables + query sketch · mock data shape · chart · target value · why it matters.

4.1 GMV (Gross Merchandise Value)

- **Source:** `payments` table (status='succeeded' OR 'released'), grouped by `created_at::date`. Filtered to **completed** jobs only — `payments.status IN ('released')` is the canonical "money actually changed hands" event.
- **Per-metro:** join `payments → job_milestones → jobs → hubs.region` for metro slice.
- **Per-trade:** join `payments → jobs → ... → pro_trades.tradeCategory` (LATERAL on accepted bid → pro).
- **Query sketch (totals window):**

```sql
SELECT date_trunc('day', released_at) AS day,
       SUM(amount_cents) AS gmv_cents
  FROM payments
 WHERE status = 'released'
   AND released_at >= NOW() - INTERVAL '90 days'
 GROUP BY 1 ORDER BY 1;
```

- **Mock shape:** `[{ day: '2026-04-22', gmv: 12450, metro: 'Portsmouth', trade: 'Electrical' }, ...]`
- **Chart:** Tremor `AreaChart` with stacked metro layers; window selector tabs (7d/30d/90d) re-fetches.
- **Target investor wants to see:**
  - Phase 0 exit: any non-zero, ideally **$10K+ in beta GMV by W12**
  - Phase 1 exit: **$200K+ GMV** (per spec §12)
  - Phase 2 exit: **$1M+ annualized**
- **Why it matters:** GMV is the single most-asked-for marketplace number. Without GMV the deck is theory. With even small GMV the deck becomes a traction story.

4.2 Take-rate capture

- **Source:** `payments.commissionCents` summed over `released_at` window.
- **Beta vs standard split:** beta cohort pros are flagged via `pros.badgeTier = 'founding'` (proposed — see §6 assumptions) OR a new `pros.is_founding_pro` boolean (preferred). For Phase 0, mock a hard-coded list of beta pro IDs.
- **Query sketch:**

```sql
SELECT date_trunc('week', released_at) AS week,
       SUM(commission_cents) FILTER (WHERE p.is_founding) AS beta_take_cents,
       SUM(commission_cents) FILTER (WHERE NOT p.is_founding) AS
standard_take_cents
  FROM payments pay
  JOIN bids b ON b.job_id = pay.job_id AND b.status = 'accepted'
  JOIN pros p ON p.id = b.pro_id
 WHERE pay.status = 'released'
   AND pay.released_at >= NOW() - INTERVAL '90 days'
 GROUP BY 1 ORDER BY 1;
```

- **Mock shape:** `[{ week: '2026-W17', betaTake: 250, standardTake: 0 }]`
- **Chart:** Tremor `LineChart` two series, $ on Y axis.
- **Target investor wants to see:** beta line growing; standard line appearing post-beta (M4+) without a take-rate-cap collapse.
- **Why it matters:** Proves the unit economics. 5% × $200K Phase 1 GMV = $10K commission — small in absolute, big as proof of mechanism. Take-rate is the durability metric for the Series A.

4.3 Pro liquidity ratio (% jobs matched <2hr)

- **Source:** `jobs.created_at` vs first `dispatch_attempts.notified_at` per job; "matched" = `dispatch_attempts.response = 'accept'`.
- **Per-metro:** join via `jobs.hubId → hubs.region`.
- **Query sketch:**

```sql
SELECT h.region AS metro,
       COUNT(*) FILTER (
         WHERE EXTRACT(EPOCH FROM (first_accept.notified_at - j.created_at))
<= 7200
       )::float / NULLIF(COUNT(*), 0) AS pct_matched_2hr
  FROM jobs j
  LEFT JOIN hubs h ON h.id = j.hub_id
  LEFT JOIN LATERAL (
    SELECT MIN(notified_at) AS notified_at
      FROM dispatch_attempts da
     WHERE da.job_id = j.id AND da.response = 'accept'
  ) first_accept ON TRUE
 WHERE j.created_at >= NOW() - INTERVAL '30 days'
 GROUP BY h.region;
```

- **Mock shape:** `[{ metro: 'Portsmouth', pct: 0.71 }, { metro: 'Manchester', pct: 0.55 }, ...]`
- **Chart:** Tremor `BarChart` horizontal, with a dashed reference line at 80% (target).
- **Target investor wants to see: 80%+** (Phase 1 gate per spec §12: "<2hr match time"). Below 50% = supply problem, above 90% = demand problem.
- **Why it matters:** Liquidity is the thing every marketplace investor (NFX especially) asks about by minute 8 of the call. <2hr is the contractual promise of "Uber for trades."

4.4 Avg bids per job

- **Source:** `bids` joined to `jobs`. Compute mean `COUNT(bids)` per job over rolling 30 days; trailing 30-day single number + sparkline.
- **Query sketch:**

```sql
WITH per_job AS (
  SELECT j.id, COUNT(b.id) AS bid_count, j.created_at::date AS day
    FROM jobs j
    LEFT JOIN bids b ON b.job_id = j.id
   WHERE j.created_at >= NOW() - INTERVAL '60 days'
     AND j.status NOT IN ('draft', 'cancelled')
   GROUP BY j.id, j.created_at
)
SELECT day, AVG(bid_count) AS avg_bids
  FROM per_job
 GROUP BY day ORDER BY day;
```

- **Mock shape:** `[{ day, avgBids: 2.4 }, ...]` plus headline scalar `2.7`.
- **Chart:** Tremor `Metric` + `SparkAreaChart`.
- **Target investor wants to see: 3–5 bids per job**. <2 = thin supply; >7 = pros wasting time / signal of low close-rate.
- **Why it matters:** It is the customer-perceived value moment. A homeowner posts a job and sees 4 bids in an hour — that is the platform working.

4.5 Pro retention cohort curve

- **Source:** `pros.joinedAt` for cohort assignment (week of signup); "active in week N" = pro had ≥1 `bid` OR ≥1 `dispatch_attempt` response in calendar week N relative to joinedAt.
- **Query sketch (W4 cohort retention — repeat for W8, W12):**

```sql
WITH cohorts AS (
  SELECT id, date_trunc('week', joined_at) AS cohort_week FROM pros
),
activity AS (
  SELECT b.pro_id, date_trunc('week', b.created_at) AS active_week FROM bids b
  UNION
  SELECT da.pro_id, date_trunc('week', da.notified_at) FROM dispatch_attempts
da WHERE da.response IS NOT NULL
)
SELECT c.cohort_week,
       COUNT(DISTINCT c.id) AS cohort_size,
       COUNT(DISTINCT a.pro_id) FILTER (
         WHERE a.active_week = c.cohort_week + INTERVAL '4 weeks'
       ) AS active_w4
  FROM cohorts c
  LEFT JOIN activity a ON a.pro_id = c.id
 GROUP BY c.cohort_week ORDER BY c.cohort_week;
```

- **Mock shape:** `[{ cohort: '2026-W12', size: 5, w4: 5, w8: 4, w12: 4 }, ...]`
- **Chart:** TanStack Table (cohort × week grid) with conditional color-fill (Tremor `Tracker` reuse) — cells color from red (0%) → emerald (100%).
- **Target investor wants to see: >80% pro retention** at W4–W12 (Phase 1+2 gate per spec §12). Below 60% = pricing/value-prop problem.
- **Why it matters:** Pro retention is the Series A gating number. If pros churn, the marketplace dies. One number, one gate.

4.6 Client repeat rate

- **Source:** `jobs.clientUserId` grouped; count clients whose 2nd job posted within 90 days of their 1st.
- **Query sketch:**

```sql
WITH client_jobs AS (
  SELECT client_user_id, created_at,
         ROW_NUMBER() OVER (PARTITION BY client_user_id ORDER BY created_at)
AS job_num
    FROM jobs WHERE status NOT IN ('draft', 'cancelled')
)
SELECT
  COUNT(DISTINCT cj1.client_user_id) AS clients_with_one,
  COUNT(DISTINCT cj2.client_user_id) AS clients_with_two_in_90d
FROM client_jobs cj1
LEFT JOIN client_jobs cj2
  ON cj2.client_user_id = cj1.client_user_id
 AND cj2.job_num = 2
 AND cj2.created_at <= cj1.created_at + INTERVAL '90 days'
WHERE cj1.job_num = 1;
```

- **Mock shape:** scalar `0.34` (34%).
- **Chart:** Tremor `ProgressBar` to a 40% target, `BadgeDelta` for week-over-week change.
- **Target investor wants to see: 30%+ within 6 months of launch**. Repeat clients are 5× cheaper to serve and the prerequisite for any LTV story.
- **Why it matters:** The competitive moat. Angi/Thumbtack actively suppress repeat (they want to keep selling leads). Sherpa Pros' thesis is to *encourage* repeat — this metric proves the model is pulling.

4.7 NPS (Pro + Client)

- **Source:** Currently NO NPS table. Need to add `nps_responses` table (proposed in §6 assumptions). For Phase 0, hand-entered weekly numbers from the 10-minute beta call go into a JSON file or a single-row admin form.
- **Mock shape:** `[{ week: '2026-W12', proNps: 60, clientNps: 45 }, ...]`
- **Chart:** Tremor `LineChart` two series, Y axis -100 to +100, reference line at 50 (Phase 1 gate).
- **Target investor wants to see: NPS >50** for both sides (Phase 1 gate, spec §12). Pro NPS lagging client NPS = supply-side neglect; the inverse = pricing too cheap.
- **Why it matters:** NPS is the only qualitative-style metric investors trust at low N. Two numbers, weekly, no excuses.

4.8 Wiseman usage telemetry

- **Source:** Currently NO usage events table. The Wiseman calls happen in `src/lib/wiseman-bridge/client.ts` and downstream in `src/lib/dispatch-wiseman/`. Need a `wiseman_events` table (proposed in §6 assumptions) capturing `{event_type, user_id, job_id, latency_ms, created_at}`.
- **Event types tracked:** `quote_validation` (bid vs. Wiseman estimate deviation check), `code_check` (NEC/IRC/MA Electrical / NH RSA validation), `scope_approval` (homeowner accepted Wiseman-generated scope of work).
- **Query sketch:**

```sql
SELECT date_trunc('day', created_at) AS day,
       event_type,
       COUNT(*) AS events
  FROM wiseman_events
 WHERE created_at >= NOW() - INTERVAL '30 days'
 GROUP BY 1, 2 ORDER BY 1;
```

- **Mock shape:** 3 series, 30 daily points each.
- **Chart:** Three Tremor `SparkLineChart` cards side-by-side. Each card: today's value + 7d delta + 30d sparkline.
- **Target investor wants to see: growing daily volume** + a healthy ratio (1 quote-validation per bid, 1 code-check per electrical/plumbing job, 1 scope-approval per accepted job).
- **Why it matters:** Wiseman is the moat (per spec §10 R7). Telemetry proves it isn't vaporware — every job actually routes through the AI layer.

4.9 Funnel (posted → reviewed)

- **Source:** All from `jobs`, `dispatch_attempts`, `bids`, `payments`, `ratings`.
- **Stages:**
  1. **Posted:** `jobs.status != 'draft'`, in 30d window
  2. **Matched:** has ≥1 `dispatch_attempts` for that job
  3. **Bid:** has ≥1 `bids` for that job
  4. **Accepted:** has ≥1 `bids.status = 'accepted'`
  5. **Completed:** `jobs.status = 'completed'`

6. **Paid:** has `payments.status = 'released'`
7. **Reviewed:** has `ratings` row

- **Query sketch:** 7 separate `COUNT(DISTINCT job_id)` queries `UNION ALL`-ed, or one CTE chain. Order matters: each stage's denominator is the prior stage's numerator.
- **Mock shape:** `[{ stage: 'Posted', count: 30 }, { stage: 'Matched', count: 28 }, ..., { stage: 'Reviewed', count: 14 }]`
- **Chart:** Tremor `BarList` (horizontal stepped bars; built-in decreasing visualization). Stretch goal: replace with proper Sankey post-Phase-1 using `react-flow` or `@nivo/sankey`.
- **Target investor wants to see: monotonic decline** with no drop >40% at any single stage. Cliff at "Bid → Accepted" = pricing/quality issue. Cliff at "Completed → Paid" = Stripe Connect issue. Cliff at "Paid → Reviewed" = review prompt friction.
- **Why it matters:** Funnel is where investors locate operating leverage. "Where do you lose people?" is question 11 of every marketplace pitch.

5. Headline KPI strip (Row 1)

CARD	METRIC	SOURCE	TARGET
GMV (90d)	$X total	sum payments.amountCents released_at >= NOW() - 90d	Phase 1 = $200K cumulative
Take-rate captured (90d)	$X commission	sum payments.commissionCents	5% of GMV (beta)
Active pros	count pros where onboardingStatus='active'	pros table	Phase 0 = 10+, Phase 1 = 50+
NPS (latest week)	Pro X / Client Y	latest nps_responses row	>50

Each uses Tremor `Metric` + `BadgeDelta` for week-over-week. Sparkline beneath via `SparkAreaChart`.

6. Schema fields assumed (NOT in current `drizzle-schema.ts`)

The current schema is sufficient for ~60% of the dashboard. The following additions are needed for the rest. **Coordinator: please verify and add migrations before live data flows.**

FIELD / TABLE	USED BY WIDGET(S)	WHY
`pros.is_founding_pro: boolean` (or extend `badgeTier` enum to include `'founding'`)	4.2 Take-rate split	Identifies the beta-cohort 5% take-rate pros vs. standard 10% pros.
`nps_responses` table: `id, user_id, role ('pro'|'client'), score (0-10), comment, created_at`	4.7 NPS, Row 1 KPI	NPS data has nowhere to live today.
`wiseman_events` table: `id, event_type, user_id, job_id, latency_ms, payload jsonb, created_at`	4.8 Wiseman telemetry	No telemetry capture exists today.
`jobs.matched_at: timestamp` (denormalization) — convenience column set when first `dispatch_attempts.response='accept'` lands	4.3 liquidity, 4.9 funnel	Avoids LATERAL joins on every dashboard refresh. Optional.
`hubs.metro_label: varchar` — human-readable metro name (current `region` is short code like 'NH')	All per-metro breakouts	"Portsmouth", "Manchester", "Portland", "Boston" labels for charts.

The implementation page below uses the schema as-is, with TODO comments at every spot where these additions would replace the mock.

7. Auth gate

`requireRole()` only supports `pro | client | pm | tenant`. There is no "admin" role yet. Two options:

1. **Quick (Phase 0):** Hard-code Phyrom's Clerk user ID in env `ADMIN_USER_IDS` (comma-separated) and gate via a small `requireAdmin()` helper that wraps `currentUser()` and checks `userId IN ADMIN_USER_IDS`. Implementation includes this helper inline in the page (so we don't modify files outside the allowed scope).
2. **Proper (Phase 1+):** Add `'admin'` to `UserRole` union and wire through `roles.ts`, `require-role.ts`. Defer.

Implementation uses option 1, with a TODO note for the coordinator.

8. Refresh & caching

- Server Component default — re-renders on each request.
- Add `export const revalidate = 300;` (5-min ISR) so investor link doesn't hammer the DB.
- Manual refresh button in header (forces `router.refresh()`).
- Heavy queries can be wrapped in `unstable_cache` later if needed.

9. Phased rollout — recommended priority order for replacing mock with live data

As beta cohort data starts flowing, swap mocks in this order (highest signal first):

1. **GMV time-series** (4.1) — first dollar through Stripe Connect = first real chart. Day-1 demand from investors.
2. **Active pros count** (Row 1) — already queryable from `pros` today; trivial to wire live.
3. **Funnel** (4.9) — purely derived from existing tables; biggest "is this real" signal for investors.
4. **Avg bids per job** (4.4) — also queryable today; small N is fine, the trendline matters.
5. **Pro liquidity ratio** (4.3) — needs `dispatch_attempts` data, which flows the moment real jobs route.
6. **Take-rate capture** (4.2) — needs `pros.is_founding_pro` migration first; then trivial.
7. **Client repeat rate** (4.6) — meaningful only at N>30 jobs. Mock until M2.

8. **Pro retention cohort** (4.5) — meaningful only at N > 4 weekly cohorts. Mock until M2.
9. **NPS time series** (4.7) — needs `nps_responses` table + weekly survey send (Resend or Twilio). Migration first.
10. **Wiseman usage telemetry** (4.8) — needs `wiseman_events` table + instrumentation in `src/lib/wiseman-bridge/`. Migration + dev work — defer to Phase 1.

10. Open questions for Phyrom

1. Should the dashboard be **public** (link in data room, anyone with URL can view) or **gated** (Clerk-authed admin only)? Implementation defaults to **gated**; if public, strip admin-layout chrome and add a `/public-traction` route variant.
2. **Per-trade breakout** — granularity? Group by 7-category bucket (Plumbing/Electrical/HVAC/Carpentry/etc.) or by full 37-trade list?
3. **Investor-friendly currency formatting** — show $1.2K, $12.4K, $1.2M short form? Implementation uses short form (more skim-able).
4. **Phase 0 gate progress widget** — should the footer literally show "$X of $250K committed" with a progress bar? Implementation includes a placeholder; needs real funding-tracker data source.

Appendix A — File map

- Page: `src/app/(dashboard)/admin/investor-metrics/page.tsx`
- Mock generators (inline in page, top of file)
- Live queries (inline; will migrate to `src/db/queries/investor-metrics.ts` once data flows)
- This design doc: `docs/pitch/metrics-dashboard-design.md`